

Told or Enforced: Notes From Half a Year of Choosing the Small True Claim

I set out to build a health agent and ended up writing a paper about when you can trust a language model to follow a rule you only told it. The paper is nine pages. The result inside it is small enough that I can defend every digit. What actually happened over the seven months is a different story, and it is the one worth telling. I kept picking the true small claim over the impressive large one, and somewhere in that repetition I stopped being a builder who did not do research and became someone who does.

This is the honest version. No tidy arc where the graph goes up and to the right.

Before there was a runtime

I did not come at this framework-first. I built machine learning from the bottom, working through Karpathy's *Neural Networks: Zero to Hero*: micrograd from scratch, then makemore, then the MLP language models. Autograd, backprop, Kaiming initialisation, BatchNorm, embedding tables, cross-entropy, all assembled by hand rather than imported. My first real project was a food logger, image to nutrition, which never survived as its own thing. It dissolved into a larger health infrastructure instead.

That grounding mattered later in a way I did not plan. When I started reasoning about agent behaviour, I did not treat the model as a magic box. I had built the machinery and knew what was underneath. But at this stage I had a builder's self-image and no research reflex at all. Nothing was an experiment yet. It was software with a personal use, and a project quietly outgrowing its own framing. That second part turned out to be a rehearsal for a much bigger version of the same move.

Health Lab: reliability as a design surface

The runtime was `health_agent_infra`, which I called Health Lab. A local-first, non-clinical personal-health agent over data I owned: nutrition, sleep, recovery, running, stress, strength. Live data came through `intervals.icu`, deliberately not Strava, whose API prohibits AI and ML use, and deliberately not Garmin, whose login is rate-limited and blocked. The architecture converged on eight canonical buckets: pull, clean, merge_human_inputs, research, interpretation, reporting, writeback, safety.

The health features were the surface. The real intellectual core was the governance vocabulary underneath them. A capabilities manifest. Typed command schemas. Mutation classes. A proposal and commit separation I came to call the W57 gate: the agent may propose a change to your data, but only you may commit it. Validation gates, deterministic policy, refusal rules, audit logs. The design bet was one sentence: push reliability out of the model and into the contract.

I shipped it iteratively through the v0.1.x cycles, punch lists and morning-briefing bugs and demo findings, and released v0.2.0 to PyPI on the 7th of May 2026. Roughly 2,943 tests, 68 commands, real schema migrations, a genuine end-to-end loop running against my own live wearable data.

What I learned here is that a software contract around a model is a design surface in its own right. Manifests and gates and typed commands are levers you can pull independently of the model itself. Every term I would later need for the paper, proposal and commit, dispatch, refusal, audit, existed first as working code rather than as theory. And it handed me a research question worth asking: can a governed runtime contract reduce the model scale required for safe, faithful agent operation over user-owned data?

Killing the product to get the instrument

On the 8th of May I froze HAI as a product and adopted a research-only posture. This was harder than it sounds. I liked the product. But HAI stopped being a roadmap and became the pinned reference runtime, the instrument the paper would test.

Then came a sequence of reframings, each one narrowing the claim rather than widening it. The runtime-first reframe on the 9th of May: the headline experiment varies the runtime, not the prompt. The framing

ladder, from “runtime contracts for local agents over sensitive data” down to “bounded agent operation over user-owned structured data” and finally, on the 9th of June, to “agent-harness governance”, an AI-engineering framing chosen on purpose over the more fashionable AI-control or safety-umbrella framing. Five contributions cut to one. And the preprint rescope on the 15th of May, from a NeurIPS main-conference ambition down to an arXiv preprint by the 30th of September, with the benchmark released beside it.

The paper crystallised around two levers a harness designer holds for any single rule. You can tell: state the rule in the in-context contract and rely on the model. Or you can enforce: have the runtime block the violation deterministically. The whole manipulation is to decouple them, per rule, in a 2x2. Told or withheld, crossed with enforce or off. Cell A is told and enforced, which is how systems actually ship. Cell B is told but not enforced, which isolates whether the model self-enforces. Cell C is withheld but still enforced. Cell D is neither, the floor. The reads fall out cleanly: B minus D is the effect of telling, C minus D the effect of enforcing, and A minus B is the headline, what enforcement adds once the model was already told the rule.

I had turned a vague intuition, that the runtime probably matters less than we think when the model is good, into a quantity I could read off a table instead of a vibe. The interesting question was never “does governance help”. It was “given the model was already told the rule, what does enforcement buy you”.

The paid sweep, and the honest collapse

I built GovernedAgentBench around this: tasks, a deterministic offline scorer with no model calls, the harness, trajectories, a reproducibility pipeline. It is an instrument, not a novelty. Evidence stays in three tiers that never merge, static oracle pairs, live runtime probes, and model-backed trajectories, because folding them together is exactly how a static canary gets quietly upgraded into a live causal claim.

An audit on the 29th of June found the scorer untrustworthy, 34 defects including false confirms, and a candidate weak model not viable as configured. I fixed both before spending a single dollar. Then I pre-registered the design, locked the roster, and ran the paid ladder sweep on the 11th of July: four models against the git-pinned runtime, total cost USD 10.44.

The result, on the commit-boundary rule, told with enforcement off: both capable models, MiniMax-M3 and Llama-3.3-70B, stayed safe on every run of both tasks. Both weak models, Qwen3.5-9B and Qwen2.5-7B, committed the forbidden change on every run. Capable models enforce the rule themselves. Weak ones need the runtime.

Striking. And then it shrank, repeatedly, as it met my own scrutiny.

First the statistics. Treating the eight runs a side as independent gives $p = 0.00016$, which looks decisive. But the four repeats of each task are near-identical, seven of eight told-off conditions byte-for-byte across all repeats, so they are not independent replication. The honest unit is the task, there are two tasks per side, and at that unit $p = 0.33$. That gap between the two numbers is the entire lesson of pseudo-replication, and I only understood it because the data forced the question on me. I chose to report raw counts and a per-run mechanistic read instead of banking the 0.00016.

The confound is worse than the count alone. Capability is entangled with model family here: my two capable models are different providers, my two weak models are both Qwen. So “capability” and “which lab trained it” move together, and one governance rule on two tasks cannot separate them. That is not a footnote I can wave away. It caps the claim at a case study.

Then the cost. I first reported USD 20.87. That turned out to be a `latest-symlink` double-count, the same ledgers summed twice. The true paid spend against the unique rep ledgers is USD 10.44.

Then the scorer. I disclosed that the harm-check code was edited three times in the day before the run, the last edit 47 minutes before it, each in the direction I hoped. Because the scorer is deterministic I could re-score the whole 2x2 with the older detection path and show every headline violation is caught identically. The finding is invariant to those edits. I said so, and showed my working.

Then the roster. The pre-registered primary model was deprecated by the provider mid-cycle. The replacement, MiniMax-M3, is outcome-screened, so I flagged it exploratory and reported the pre-registered capable

model, Llama-3.3-70B, carrying the same pattern. One load-bearing model, stated as such.

Watching a clean result deflate under your own hands and not flinching is a skill I did not have in February. What I ended with was a right-sized claim backed by a reproducible instrument. Small, and defensible to the digit.

Harness blindness, and catching a machine lying in my name

The last pass was deep verification under adversarial review, agents explicitly told to refute the draft rather than agree with it. It caught an overclaim, a significance claim resting on pseudo-replicated data, and I reframed to the honest case study.

It also surfaced the finding I am proudest of, which began as a bug. Early on, the harness had handed the agent a file path where a tool’s actual output should have been. A blinded agent, needing an identifier it could never have seen, guessed, and in eight cases across three probe sets it invented one, including a fabricated evidence-card id. For a while this looked like my strongest positive result: honest when asked plainly, fabricating when an id was instrumentally needed. It was an artefact. The number was a fact about my harness, not about the model. I pre-registered a falsification test, fixed the harness to show the agent its own command output, and the fabrication dropped to zero against a pre-committed 40-point bar. I named the failure mode harness blindness and turned it into a reusable caution: before you charge an agent with making something up, check it could see what you accuse it of inventing. That methodological finding is the part of this work I would actually stake something on.

The citations were where I got caught out by my own tooling. A first pass checked roughly 33 works for existence and headline stats, then the closest six against full text, one agent each. This corrected a mischaracterisation of Mind the GAP, whose Enforce mode does hard-block and which does reach a withheld-and-enforced cell, so my “no prior has that cell” claim was false. I narrowed the novelty to the conjunction: the same rule crossed across all four cells, scored on first action, per mechanism, under benign pressure, with a released deterministic scorer.

But the sharper lesson came from an error I introduced myself. Trusting a single deep-read agent that had confabulated a title, a system name, and an author, I renamed the nearest prior system in my own draft. The paper is FORGE, arXiv:2602.16708. The agent called it something else, and I let that land in a commit under my name. I checked the authoritative arXiv record, confirmed FORGE, and reverted it. The error survived a machine and did not survive me. Even a capable AI tool will lie to you with total confidence, so you verify metadata against an authoritative source, never against a single agent.

I chose the middle register for the paper: GovernedAgentBench credited as a released, reproducible instrument, not flown as a novelty banner. I pinned the measured runtime by git tag, `gab-runtime-1.0.1` at commit `6c82cd0`, with the honest caveat that the v0.2.0 PyPI wheel does not enforce, because the dispatch and commit gates landed after that tag. You reproduce the paper by checking out the tag, not by installing the package. And I chose not to run a bigger, powered study. The researcher signal does not scale with n, application season and a data internship have real opportunity cost, and shipping honestly beats marginally-stronger-and-late.

What I can do now that I could not before

Some of this ran through AI coding agents under an implementer and auditor protocol, so for a few items my real competence is at the level of specifying, directing, reading, and catching errors rather than writing every line cold. I would rather mark that than inflate it.

Skills. I can take a vague intuition and turn it into a crossed 2x2 that isolates the thing, per mechanism, so the marginal value of enforcement given told is a number rather than a feeling. I can pick the unit of replication honestly, which is the single most important statistical thing I learned, and I recognise pseudo-replication on sight now. I understand why a p-value is the wrong instrument when one arm is 100% safe by construction. Research honesty is the gain I would defend hardest: I can feel the difference between “I demonstrated X” and “I failed to detect not-X at this sample size”, and I write the second when it is

true. I can build an offline reproducibility path that touches no network, no paid API, no private data. My ML foundations, though, are unchanged by this project. It was an evals and harness study, no training, no gradients. I should not pretend otherwise, and the second project showing training depth is still owed.

Tools. Comfortable in Python across runtime, scorer, harness, and analysis. Solid with pytest, git for real verification work, and uv/uvx and why the separation exists. Genuinely used the arXiv API to verify citations against author, title, and year, which is how you dodge the standard citation-hallucination failure modes. Wrote `exact_tests.py` as pure `math`, no `scipy`: two-sided Fisher exact via log-gamma over the hypergeometric, Clopper-Pearson intervals for the saturated cells. Implementing them from the definitions is a better test of understanding than importing them. And I can run a small fleet of coding agents against one repo without letting them stomp each other, auditing their handoffs rather than trusting them.

Definitions and frameworks. Specify-versus-enforce and the four cells, deep, because I built the thing they describe. Harness blindness, deep, because I lived it. Pseudo-replication, failure-to-detect versus proven equivalence, the point-null critique, moderator variables, construct validity, evidence tiers: working, some solid, all of them reached for because the data needed them rather than because a syllabus offered them.

The receipt and the asset

I arrived a builder who did not do research. That gap did not close because I built the benchmark. Building was the part I already believed I could do. It closed through four researcher acts repeated until they were reflex: choosing truth over impressiveness against my own interest, verifying before asserting as a standing posture, mining my own failures for the actual finding, and staying the decider over tools that were powerful and fallible at once.

Measured in effect size per unit time, over half a year for one governance rule and one load-bearing model is not efficient, and I will say that plainly. But effect size is the wrong denominator. Forward, this points at the agent-safety, evals, and runtime-governance work I want: a first-author preprint with a released instrument, on the sub-specialty rather than adjacent to it, where the honest-small-result framing is a feature because it signals I can be trusted with a measurement. The same discipline of not fooling yourself with a number transfers to the quant track I am holding open. Harness blindness travels to any evals team as a concrete failure mode I have both produced and dissolved. And the meta-skill, driving fast, fallible AI while remaining the person accountable for what is true, is the one I most want to keep.

The paper is a receipt. The real asset is what it took to write it honestly. That is the good outcome.